Live Translation: Bun, LiveKit and Python

A practical summary of how Mawiid should connect its existing Python speech and translation models to LiveKit rooms, while keeping Bun responsible for application security and room management.

Decision

Use the existing Python service for real-time captions and translation. Keep Bun as the application backend. Do not use VoltAgent for the audio translation worker. If a JavaScript worker is ever needed, use LiveKit Agents for Node.js instead, because it is designed to join rooms and process realtime media.

The architecture

Frontend	Bun backend	LiveKit	Python worker
Joins the room and displays captions.	Authenticates users, issues tokens, stores room settings, dispatches the agent.	Carries audio, room events, and targeted caption messages.	Receives audio, transcribes, translates, and publishes captions.

There is no direct browser-to-Python connection. Bun and Python can use different programming languages because both communicate with LiveKit using its server and realtime room APIs.

End-to-end meeting flow

1. The host enables translation. Bun verifies the role, stores the setting, and synchronizes the room metadata to LiveKit.

2. Bun explicitly dispatches **mawiid-translator**. The Python worker joins the LiveKit room as a system participant only when required.

3. The worker subscribes to human microphone tracks and runs voice activity detection, Faster-Whisper speech-to-text, and MADLAD translation for completed phrases.

4. It groups listeners by target language, translates once per target, and sends each caption only to the relevant participant identities.

5. The frontend displays the caption matching the selected language. Bun persists final transcript and translation segments through a protected internal endpoint.

6. When translation is disabled or the room ends, Bun deletes the dispatch and the worker releases its audio and model resources.

Supported language contract

The Python pipeline accepts exactly these six locale codes. Bun must validate and normalize user language values to this list before issuing LiveKit tokens or updating participant attributes.

Supported locales: **en-US** English, **fr-FR** French, **ar-SA** Arabic, **ar-MA** Moroccan Darija, **zh-CN** Simplified Chinese, and **zh-TW** Traditional Chinese. Chinese variants share the Mandarin model output, then use the selected script. Normalize Bun's current **en** fallback to **en-US** before issuing tokens.

Implementation checklist

1	Worker runtime	Add a LiveKit Python worker registered as mawiid-translator.
2	Dispatch lifecycle	Bun creates or removes one idempotent dispatch when the room setting changes.
3	Shared contract	Add mawiid.translation.v1: speaker, source and target locale, timing, text, and partial/final state.
4	Frontend captions	Render targeted translation data. Never expose worker credentials in the browser.
5	Persistence	Python sends final segments to an authenticated Bun endpoint, which writes to the tenant vault.
6	Operations	Run separately from Bun with server-only credentials and WSS access to the same LiveKit project.

Security and deployment rules

1. Only Bun and Python receive LiveKit API credentials; the frontend receives short-lived participant tokens.

2. Use LiveKit Cloud or a public LiveKit server with WSS and TURN. A frontend Cloudflare tunnel alone does not publish local media.

3. Use targeted data messages, and log operational metadata only - never tokens or raw audio.